Efficient Three-party Boolean-to-Arithmetic Share Conversion

Nan Cheng
School of Computer Science
University of St. Gallen
St. Gallen, Switzerland
nan.cheng@unisg.ch

Feng Zhang
School of Computer and Software
Nanyang Institute of Technology
Nanyang, China
gudengxia@gmail.com

Aikaterini Mitrokotsa
School of Computer Science
University of St. Gallen
St. Gallen, Switzerland
aikaterini.mitrokotsa@unisg.ch

***Abstract*—The advantage of mixed-protocol multi-party secure computation frameworks lies in their ability to utilize different sharing types optimally for diverse tasks. A key module in these frameworks are Boolean-to-arithmetic secret sharing conversion protocols, which transfer a secret value from Boolean secret sharing to arithmetic secret sharing. This conversion process can either take in the Boolean secret sharing of a bit or a secret binary string. In this work, we suggest the application of an innovative correlated random tuple for this task in the semi-honest three-party (3PC) setting. This tuple provides the basis for building Boolean-to-arithmetic share conversion protocols. Specifically, we propose two such protocols in the semi-honest 3PC setting, the first protocol takes as input the Boolean secret sharing of a bit, and the second protocol takes as input the Boolean secret sharing of a secret binary string. When it comes to concrete efficiency, the first protocol shows superior performance compared to the existing state-of-the-art in ABY3 (CCS '18). It achieves this by reducing the total required communication from 2ℓ bits per party to $4\ell/3+1$ bits (including $4\ell/3$ bits in the setup phase, and 1 bit in the online phase) per party, while maintaining a single round of optimized communication. On the other hand, the second protocol involves two rounds of online communication and its communication cost is comparable to that of ABY2.0 (USENIX'21) that relies on correlated oblivious transfer.**

***Index Terms*—Replicated Secret Sharing, Correlated Random Tuple, Boolean-to-Arithmetic Share Conversion**

I. Introduction

Secret sharing is a powerful cryptographic primitive that has been extensively employed in the secure multi-party computation (MPC). Secret sharing schemes primarily fall into three categories: Arithmetic, Boolean, and Yao's secret sharing. Share conversion protocols, which enable the transformation of secret shares, have gained considerable attention in secure MPC. These protocols facilitate the blending of different MPC protocols, thus leveraging the strengths of each. In this work, we focuse on Boolean to arithmetic secret sharing conversion protocols, we use the term *arithmetic secret sharing* when refering to secret sharing over a ring $\mathbb{Z}_{2^\ell}$ which is indicated as either $[\cdot]^{\mathsf{A}}$ or $[\![\cdot]\!]^{\mathsf{A}}$, while *Boolean secret sharing* is used to indicate secret sharing over a ring $\mathbb{Z}_2$ which is indicated as either $[\cdot]^{\mathsf{B}}$ or $[\![\cdot]\!]^{\mathsf{B}}$. We distinguish between two types of Boolean to arithmetic secret sharing conversion: we use the notation Bit2A to denote the conversion over a secret bit $b \in \{0,1\}$, and B2A for the conversion of a secret binary string $x \in \mathbb{Z}_{2^\ell}$.

The application of Boolean-to-arithmetic sharing conversions is employed primarily in two areas: (i) as essential building blocks in the construction of mixed MPC protocol frameworks [1]–[5], applicable to general secure computation tasks or more specifically, privacy-preserving machine learning; (ii) they can be employed for secure aggregation in an environment with n clients and a few servers. In this setup, each client submits the Boolean secret sharing of their private data to the servers. With Boolean secret sharing, data providers are naturally inhibited from manipulating their input data to bias the computation of the aggregation statistics, as highlighted in Prio+ [6].

A. Related work

ABYs [1]–[3] represent a significant sequence of research works addressing the transition among different secret sharing schemes, including B2A, A2B, Y2A, A2Y, B2Y, and Y2B. The principal distinction between ABY [1] and ABY2.0 [2] lies on their reliance on distinct sharing types, leading to different strategies for the Boolean-to-arithmetic conversion. For a secret input $b \in \{0,1\}$, ABY derives from the well-recognized 2-out-of-2 Boolean secret sharing indicated as $[b]^{\mathsf{B}}$, where for each $i \in \{0,1\}$ that party P_i holds share $[b]^{\mathsf{B}}_i$, satisfying $[b]^{\mathsf{B}}_0 \oplus [b]^{\mathsf{B}}_1 = b$. In contrast, ABY2.0 employs the optimized 2-out-of-2 Boolean secret sharing indicated as $\langle b \rangle^B$, where for each $i \in \{0,1\}$ that party P_i holds $\langle b \rangle^B_i = ([\delta_b]^{\mathsf{B}}_i, \Delta_b)$, where δ_b is a random bit and $\Delta_b = \delta_b \oplus b$ known by both parties. Despite the differences, both ABY [1] and ABY2.0 leverage correlated OT (cOT) [7], a form of oblivious transfer (OT), for the B2A conversion. Compared to ABY, ABY2.0 utilizes a different setup and requires less communication overhead during the online phase. Unlike ABY and ABY2.0 which operate in a two-party setting, ABY3 [3] proposed a mixed-sharing secure computation framework in the three-party scenario. This framework, built upon replicated secret sharing (RSS), can be applied to privacy-preserving machine learning. In ABY3, the authors propose realizing the Bit2A conversion using a novel three-party OT technique, and they address the B2A conversion by repetitively using a full-adder circuit within non-constant communication rounds.

An alternative approach realizing the Boolean-to-Arithmetic conversion involves pre-preparing correlated randomness dur-

ing the setup phase, resulting in a comparatively lightweight online evaluation afterwards. In these schemes the main workload lies in the setup phase, one of them is Prio+ [6], where the authors addressed the Bit2A conversion using daBit [8] prepared in the setup phase. The conversion is illustrated in a semi-honest two-party setting, which transforms $[b]^{\mathsf{B}}$ to $[b]^{\mathsf{A}}$ for a secret input $x \in \mathbb{Z}_2$: given a random daBit $([r]^{\mathsf{B}}, [r]^{\mathsf{A}})$ generated in the setup phase, in the online phase a masked bit $\delta = r \oplus b$ is revealed, then $[b]^{\mathsf{A}}$ is computed as $\delta \oplus r = \delta + [r]^{\mathsf{A}} - 2\delta \cdot [r]^{\mathsf{A}}$ locally. The notion daBit here is short for double-authenticated bit, which creates doubly authenticated bit in both Boolean and arithmetic sharing formats, primarily for use in a malicious setting. Recent research has introduced effective methods for the daBit generation [9]–[11]. To further decrease the overall generation and computation costs, the notion edaBit was proposed in [12]. It is defined as a set of m random bits $(r_{m-1}, \cdots, r_0)$, secret-shared in the binary domain, along with the arithmetic sharing of $r = \sum_{i=0}^{m-1} r_i \cdot 2^i$.

B. Our Contributions

In this work, we present two protocols Π_1, Π_2 for converting Boolean secret sharing to arithmetic secret sharing in the semi-honest three-party (3PC) setting. Our protocols are built on the use of a novel correlated random tuple, prepared in the setup phase. Given the comparatively modest cost of generating the correlated random tuple during the setup phase, coupled with the lightweight cost incurred during the online phase, we are successful in realizing a highly efficient conversion protocol. In detail, within the semi-honest 3PC setting,

- we propose a Bit2A protocol, Π_1, which incurs a cost of $4\ell/3$ bits per party in the setup phase and one bit per party in the online phase;
- we propose a B2A protocol, Π_2, requiring $(\ell^2 + 5\ell)/3$ communication bits per party, alongside two rounds of online communication.

Compared to the state-of-the-art Bit2A protocol proposed in ABY3 [3] in the semi-honest 3PC setting (which employs a dual execution of three-party oblivious transfer, transmitting two messages of 2ℓ bits in total per party in the online phase), our protocol Π_1 substantially reduces the communication cost. Specifically, we trim the online communication from 2ℓ bits per party to merely one bit per party. Even taking into account the setup phase costs, the total communication decreases from 2ℓ bits to $4\ell/3 + 1$ bits per party. Crucially, we maintain the efficient one-round communication in the online phase. Furthermore, we suggest a B2A protocol Π_2, derived from Π_1. By generating ℓ instances of correlated random tuples in the setup phase, we enable an efficient online phase at the cost of $5\ell/3$ bits per party in two rounds. This is comparable with oblivious transfer based B2A conversion protocols in ABY or ABY2.0 in terms of concrete communication cost.

As for ABY3 [3], our protocol Π_1 can directly substitute their three-party oblivious transfer-based Bit2A protocol in their 3PC secure computation framework. Although our protocol Π_2 has a higher communication cost, it outperforms their B2A protocol in terms of communication rounds, thus serving as a plausible alternative in certain tasks. Our results are directly applicable to privacy-preserving computation of aggregation statistics, i.e., when multiple clients delegate the computation of joint aggregation statistics to three external servers. We briefly compare the performance of our protocols to existing methodologies (as per our knowledge) in Table I, in terms of interfaces, offline and online costs measured in communication volume. All protocols in Table I assume a semi-honest setting.

C. Technical Overview

Our protocols are constructed in the pre-processing model, which consists of a setup and an online phase. During the setup phase, independent correlated random data is prepared among the parties. These correlated random data are then used in the online phase to facilitate a more efficient online evaluation. The underlying intuition stems from the observation: $\forall \theta, \gamma, b \in \{0, 1\}$, let $v = (-1)^\theta \cdot \gamma$, $\beta = b \oplus \gamma$, $\sigma = b \oplus \gamma \oplus \theta$, it holds that

$$b = (-1)^\sigma \cdot v + \beta. \tag{1}$$

The correctness of Eq. 1 is evident as

$$\begin{aligned}(-1)^\sigma \cdot v + \beta &= (-1)^\sigma \cdot (-1)^\theta \cdot \gamma + \beta \\ &= (-1)^{\sigma+\theta} \cdot \gamma + \beta \\ &= (-1)^{\sigma \oplus \theta} \cdot \gamma + \beta \\ &= (-1)^{b \oplus \gamma} \cdot \gamma + (b \oplus \gamma)\end{aligned}$$

If $b = \gamma$, then $b \oplus \gamma = 0$, which implies $(-1)^\sigma \cdot v + \beta = \gamma = b$. Otherwise if $b = 1 - \gamma$, then $b \oplus \gamma = 1$ which implies $(-1)^\sigma \cdot v + \beta = 1 - \gamma = b$. Hence, in both cases Eq. 1 holds.

Consider a semi-honest 3PC setting where three parties $\mathsf{P}_0, \mathsf{P}_1, \mathsf{P}_2$ are divided into two groups $\mathsf{G}_0, \mathsf{G}_1$, where G_0 comprises of P_0 and G_1 comprises $\mathsf{P}_1, \mathsf{P}_2$. We find that from Eq. 1 we can realize the Bit2A conversion in the pre-processing model by completing following goals.

1) In the setup phase, the goal is to prepare a correlated random tuple $\{[\![v]\!]^{\mathsf{A}}, \gamma, \theta\}$ where $v = (-1)^\theta \cdot \gamma$, with G_0 holds $\gamma \leftarrow_{\$} \{0, 1\}$ and G_1 holds $\theta \leftarrow_{\$} \{0, 1\}$.
2) In the later online evaluation phase when the input $[\![b]\!]^{\mathsf{B}}$ is accessible, the goal is to have all parties reveal a masked bit $\sigma = b \oplus \gamma \oplus \theta$ (notice along the revelation $\beta = b \oplus \gamma$ is also revealed to G_1).

Thus, with G_1 possessing $[\![\beta]\!]^{\mathsf{A}} := \{(0, 0), (0, \beta), (\beta, 0)\}$, all parties are able to compute $[\![b]\!]^{\mathsf{A}}$ non-interactively by executing

$$[\![b]\!]^{\mathsf{A}} = (-1)^\sigma \cdot [\![v]\!]^{\mathsf{A}} + [\![\beta]\!]^{\mathsf{A}}. \tag{2}$$

To convert $[\![x]\!]^{\mathsf{B}}$ to $[\![x]\!]^{\mathsf{A}}$ where all parties hold Boolean secret sharing $\{[\![x_0]\!]^{\mathsf{B}}, \cdots, [\![x_{\ell-1}]\!]^{\mathsf{B}}\}$, a naive idea is to perform Bit2A conversion over $[\![x_i]\!]^{\mathsf{B}}$ for each $i \in [\ell]$, and then combine the results together, *i.e.,*

$$[\![x]\!]^{\mathsf{A}} = \sum_{i=0}^{\ell-1} \mathsf{Bit2A}([\![x_i]\!]^{\mathsf{B}}) \cdot 2^{\ell-1-i}. \tag{3}$$

While this idea is enough to implement B2A conversion within a single online communication round, it does not optimize

Conversion	Ref.	#Parties	Comm.[bits] Setup	Online	Total	Rounds
Bit2A	ABY2.0 [2]	2	0	$(\lambda+\ell)/2$	$(\lambda+\ell)/2$	**1**
	ABY3 [3] (OT)	3	0	2ℓ	2ℓ	**1**
	edabits-based Bit2A [13]	n	-	$t^2+2\ell+3$	-	$\log(t+1)+3$
	Our Π_1	3	$4\ell/3$	**1**	$\mathbf{4\ell/3+1}$	**1**
B2A	ABY [1]	2	$\lambda\ell/2$	$(\ell^2+\ell)/4$	$(2\lambda+1)\ell/4+\ell^2/4$	2
	ABY2.0 [2]	2	$(\lambda\ell+\ell^2)/2$	ℓ	$(\lambda/2+1)\ell+\ell^2/2$	1
	ABY3 [3]	3	-	$\ell+\ell\log\ell$	-	$1+\log\ell$
	Our Π_2	3	$\ell^2/3$	$5\ell/3$	$(\ell^2+5\ell)/3$	2

TABLE I: Cost comparison between existing protocols and our work in the semi-honest setting. Communication (Comm.) is quantified in average bits per party. Bit2A signifies the conversion from $[b]^{\mathsf{B}} \to [b]^{\mathsf{A}}$ and $[\![b]\!]^{\mathsf{B}} \to [\![b]\!]^{\mathsf{A}}$ in a 2PC/3PC setting where $b \in \{0,1\}$. B2A symbolizes the transformation from $[x]^{\mathsf{B}} \to [x]^{\mathsf{A}}$ or $[\![x]\!]^{\mathsf{B}} \to [\![x]\!]^{\mathsf{A}}$ in a 2PC/3PC setting where $x \in \mathbb{Z}_2^\ell$. Specifically in ABY2.0 [2], the corresponding conversion protocols are implmented over the optimized secret sharing, in which λ represents the security parameter used in the oblivious transfer; in the edabits-based Bit2A [13] protocol, replicated secret sharing is employed within a general (n,t) scheme, when $(n,t)=(3,1)$, it corresponds to the 2-out-of-3 replicated secret sharing scheme, which will result in a cost of $2\ell+4$ bits across 4 rounds in the online phase.

the total communication costs. To decrease the communication cost over this naive idea, we propose Π_2 that further reduces the overall communication cost by shifting one round of communication from the setup phase to the online phase. More details are provided in section III-B.

II. Preliminaries

This section presents the notations used throughout this paper. We denote by $[n]$ $(n \in \mathbb{Z}^+)$ as a set of integers ranging from 0 to $n-1$. In formulating our protocol, we use $r \leftarrow \mathsf{Prg}(s,n)$ to signify the operation of a pseudorandom generator. This generator takes in a seed $s \in \{0,1\}^\lambda$ and a desired output length parameter $n \in \mathbb{Z}^+$, subsequently generating a pseudorandom binary string $r \in \{0,1\}^n$.

A. Secret Sharing

t-out-of-t Secret Sharing. In the two-party setting, we use $[b]^{\mathsf{B}}$ to denote the t-out-of-t Boolean secret sharing, where there are t parties $\mathsf{P}_i (i \in [t])$ holds a local Boolean share $[b]_i^{\mathsf{B}}$ such that $\oplus_{i=0}^{t-1}[b]_i^{\mathsf{B}} = b$. Likewise, $[b]^{\mathsf{A}}$ represents the t-out-of-t arithmetic secret sharing, where party $\mathsf{P}_i (i \in [t])$ holds a local arithmetic share $[b]_i^{\mathsf{A}}$ such that $\sum_{i=0}^{t-1}[b]_i^{\mathsf{A}} = b$. Specifically, we symbolize SS.add and SS.sMult as addition and scalar multiplication over 2-out-of-2 secret sharings, respectively.

Replicated Secret Sharing. In this paper, we utilize the 2-out-of-3 Replicated Secret Sharing (RSS) put forth by Araki et al [14]. RSS involves three parties: P_0, P_1, and P_2. The replicated Boolean secret sharing and the replicated arithmetic secret sharing are represented by $[\![\cdot]\!]^{\mathsf{B}}$ and $[\![\cdot]\!]^{\mathsf{A}}$ respectively. For example, given a Boolean secret sharing $[\![b]\!]^{\mathsf{B}}$ where $b \in \{0,1\}$, then P_0, P_1, and P_2 hold (b_0,b_1),(b_1,b_2) and (b_2,b_0) respectively, wherein $b = b_0 \oplus b_1 \oplus b_2$.

We utilize six components of RSS throughout this paper: RSS.share, RSS.add, RSS.zero, RSS.mul, RSS.sMul, and RSS.open.

i. RSS.share inputs $x \in \mathbb{Z}_{2^\ell}$ and outputs (x_i, x_{i+1}) to P_i for $i \in \{0,1,2\}$, in which x_0, x_1 is uniformaly randomly sampled in $\mathbb{Z}_{2^\ell}$ and $x_2 = x - x_0 - x_1$.
ii. RSS.add inputs $[\![x]\!]^{\mathsf{A}}, [\![y]\!]^{\mathsf{A}}$ and outputs $[\![x+y]\!]^{\mathsf{A}}$.
iii. RSS.zero$(\mathcal{K},\ell)$ describes a non-interactive process for creating a 3-out-of-3 sharing of zero. Here, each party P_i holds random keys $(R_i, R_{(i+1) \bmod 3})$, i.e., $K = (R_0,R_1),(R_1,R_2),(R_2,R_0)$. Given a parameter ℓ, it sends a random message $r_i \in \mathbb{Z}_{2^\ell}$ to P_i for all $i \in [3]$, ensuring $\sum_{i=0}^{2} r_i = 0 \bmod 2^\ell$.
iv. RSS.mul inputs $[\![x]\!]^{\mathsf{A}}, [\![y]\!]^{\mathsf{A}}$ and outputs $[\![x \cdot y]\!]^{\mathsf{A}}$. This operation necessitates communication among the servers and costs one round.
v. RSS.sMul takes a public scalar s and $[\![x]\!]^{\mathsf{A}}$ as inputs, outputs $[\![s \cdot x]\!]^{\mathsf{A}}$. This is a non-interactive process where all parties multiply their local shares of x with s.
vi. RSS.open inputs $[\![x]\!]^{\mathsf{A}}$ and outputs the opened value x to all parties in S.

Additionally, thoughout our work we denote the procedure for securely converting a 2-out-of-2 secret sharing $[x]^{\mathsf{A}}$ to a replicated secret sharing $[\![x]\!]^{\mathsf{A}}$ as $[\![x]\!]^{\mathsf{A}} \leftarrow \mathsf{ShareConvert}(\mathcal{K}, [x]^{\mathsf{A}})$. This procedure includes two steps that are completed within one round: initially, all parties locally invoke RSS.zero$(\mathcal{K},\ell)$ which results in a 3-out-of-3 secret sharing $[r]^{\mathsf{A}} = (r_0,r_1,r_2)$ where $\sum_{i=0}^{2} r_i = 0$. Subsequently, for each $i \in [3]$ that P_i sends $r_i + x_i$ to P_{i-1}. As a result, all parties jointly obtain replicated secret sharing $[\![x]\!]^{\mathsf{A}} = \{(r_0+x_0, r_1+x_1),(r_1+x_1, r_2+x_2),(r_2+x_2, r_0+x_0)\}$. In terms of security, we simply assume RSS.zero is realized by the ideal functionality $\mathcal{F}_{\mathsf{CR}}$ as defined in Fig. 1 from [14].

III. Our protocols

In this section we first present our Bit2A ($[\![b]\!]^{\mathsf{B}} \to [\![b]\!]^{\mathsf{A}}$) protocol Π_1, then B2A ($[\![x]\!]^{\mathsf{B}} \to [\![x]\!]^{\mathsf{A}}$) protocol Π_2. Before proceeding to the actual secure computation protocol, we formally describe the ideal functionality of $\mathcal{F}_{\mathsf{Bit2A}}$ in Fig. 2 and $\mathcal{F}_{\mathsf{B2A}}$ in Fig. 3.

A. Bit2A *Protocol*

Inspired by Eq. 1, we propose Π_1 in Protocol 1 that carries out Bit2A, *i.e.,* converting a Boolean secret sharing $[\![b]\!]^{\mathsf{B}}$ where $b \in \{0,1\}$ to an arithmetic secret sharing $[\![b]\!]^{\mathsf{A}}$ represented in ring $\mathbb{Z}_{2^\ell}$, assuming a semi-honest 3PC setting. In Protocol 1,

Let $F : \{0,1\}^* \times \{0,1\}^* \to \mathbb{Z}_{2^\ell}$ be a keyed function. Upon invocation, $\mathcal{F}_{\mathsf{CR}}$ chooses a pair of keys $k, k' \in \{0,1\}^\lambda$ and sends them to the adversary controlling party P_i. Then:

- Upon receiving input id from all parties, functionality $\mathcal{F}_{\mathsf{CR}}$ computes $\alpha_i = F_k(id) - F_{k'}(id)$ and chooses random values $\alpha_{i-1}, \alpha_{i+1} \in \mathbb{Z}_{2^\ell}$ under the constraint that $\alpha_1 + \alpha_2 + \alpha_3 = 0 \bmod 2^\ell$. $\mathcal{F}_{\mathsf{CR}}$ sends α_j to P_j for every j.

Fig. 1: FUNCTIONALITY $\mathcal{F}_{\mathsf{CR}}$.

1) $\mathcal{F}_{\mathsf{Bit2A}}$ receives b_i from P_i for each $i \in [3]$, then computes $b = \oplus_{i=0}^{2} b_i$.
2) $\mathcal{F}_{\mathsf{Bit2A}}$ chooses $v_0, v_1 \leftarrow_\$ \mathbb{Z}_{2^\ell}^2$ and computes $v_2 = b - v_0 - v_1$.
3) $\mathcal{F}_{\mathsf{Bit2A}}$ returns (v_i, v_{i+1}) to P_i for each $i \in [3]$.

Fig. 2: FUNCTIONALITY $\mathcal{F}_{\mathsf{Bit2A}}$.

we employ a pre-processing framework to achieve optimized online cost (including communication volume and communication rounds). We highlight the objectives and the respective techniques employed to achieve them in each phase.

- In the setup phase, our objective is to compute a secret arithmetic sharing $[\![v]\!]^{\mathsf{A}}$ where $v = (-1)^\theta \cdot \gamma$. Here, P_0 inputs $\gamma \leftarrow_\$ \{0,1\}$, and both P_1 and P_2 input $\theta \leftarrow_\$ \{0,1\}$. To attain this goal, we first direct P_0 to select $\gamma \leftarrow_\$ \{0,1\}$. Following this, P_0 distributes a 2-out-of-2 secret sharing $[\gamma]^{\mathsf{A}} := (\alpha_1, \gamma - \alpha_1)$ to P_1 and P_2, where $\alpha_1 \leftarrow_\$ \mathbb{Z}_{2^\ell}$. Upon receipt, P_1 and P_2 compute $[v]^{\mathsf{A}} = ((-1)^\theta \cdot \alpha_1, (-1)^\theta \cdot (\gamma - \alpha_1))$ locally as both P_1 and P_2 know the value of θ. Continually, to covert $[v]^{\mathsf{A}}$ to $[\![v]\!]^{\mathsf{A}}$, we invoke $\mathsf{ShareConvert}$ once which necessitates an additional communication round, as illustrated in line 6 of Π_1.
- In the online phase, our objective is to utilize the correlated random tuple $\{\gamma, \theta, [\![v]\!]^{\mathsf{A}}\}$, generated in the setup phase, to convert $[\![b]\!]^{\mathsf{B}}$ to $[\![b]\!]^{\mathsf{A}}$. Here, P_0 holds γ, both P_1 and P_2 hold θ, and jointly all parties hold $[\![v]\!]^{\mathsf{A}}$ where $v = (-1)^\theta \cdot \gamma$. To attain this, we apply Eq. 2, thus, we need to open both β and σ. This can be accomplished with the expenditure of one bit per party within a single round, *i.e.,*
  - let P_0 send $m_0 = b_0 \oplus b_1 \oplus \gamma$ to $\{\mathsf{P}_1, \mathsf{P}_2\}$, so that $\{\mathsf{P}_1, \mathsf{P}_2\}$ are able to compute β and σ;
  - in parallel, let P_1 send $m_1 = \theta \oplus b_2$ to P_0, with which P_0 is able to compute σ.

  After all parties obtain σ, we represent $[\![\beta]\!]^{\mathsf{A}} := \{(0,0), (0,\beta), (\beta,0)\}$ (notice only $\{\mathsf{P}_1, \mathsf{P}_2\}$ have the information of β), then all parties are able to compute $[\![b]\!]^{\mathsf{A}}$ locally as follows:

1) $\mathcal{F}_{\mathsf{B2A}}$ receives x_i from P_i for each $i \in [3]$, then computes $x = \oplus_{i=0}^{2} x_i$.
2) $\mathcal{F}_{\mathsf{B2A}}$ chooses $v_0, v_1 \leftarrow_\$ \mathbb{Z}_{2^\ell}^2$ and computes $v_2 = x - v_0 - v_1$.
3) $\mathcal{F}_{\mathsf{B2A}}$ returns (v_i, v_{i+1}) to P_i for each $i \in [3]$.

Fig. 3: FUNCTIONALITY $\mathcal{F}_{\mathsf{B2A}}$.

γ	θ	$[\![v]\!]^{\mathsf{A}}$	b	β	σ	$[\![v^*]\!]^{\mathsf{A}}$	$[\![v^*]\!]^{\mathsf{A}} + [\![\beta]\!]^{\mathsf{A}}$
0	0	$[\![0]\!]^{\mathsf{A}}$	0	0	0	$[\![0]\!]^{\mathsf{A}}$	$[\![0]\!]^{\mathsf{A}}$
0	0	$[\![0]\!]^{\mathsf{A}}$	1	1	1	$[\![0]\!]^{\mathsf{A}}$	$[\![1]\!]^{\mathsf{A}}$
0	1	$[\![0]\!]^{\mathsf{A}}$	0	0	1	$[\![0]\!]^{\mathsf{A}}$	$[\![0]\!]^{\mathsf{A}}$
0	1	$[\![0]\!]^{\mathsf{A}}$	1	1	0	$[\![0]\!]^{\mathsf{A}}$	$[\![1]\!]^{\mathsf{A}}$
1	0	$[\![1]\!]^{\mathsf{A}}$	0	1	1	$[\![-1]\!]^{\mathsf{A}}$	$[\![0]\!]^{\mathsf{A}}$
1	0	$[\![1]\!]^{\mathsf{A}}$	1	0	0	$[\![1]\!]^{\mathsf{A}}$	$[\![1]\!]^{\mathsf{A}}$
1	1	$[\![-1]\!]^{\mathsf{A}}$	0	1	0	$[\![-1]\!]^{\mathsf{A}}$	$[\![0]\!]^{\mathsf{A}}$
1	1	$[\![-1]\!]^{\mathsf{A}}$	1	0	1	$[\![1]\!]^{\mathsf{A}}$	$[\![1]\!]^{\mathsf{A}}$

TABLE II: Truth Table of Π_1.

$$[\![v^*]\!]^{\mathsf{A}} \leftarrow \mathsf{RSS.sMul}((-1)^\sigma, [\![v]\!]^{\mathsf{A}})$$
$$[\![b]\!]^{\mathsf{A}} \leftarrow \mathsf{RSS.add}([\![v^*]\!]^{\mathsf{A}}, [\![\beta]\!]^{\mathsf{A}})$$

Protocol 1 Bit2A potocol (Π_1)

Functionality: $[\![b]\!]^{\mathsf{A}} \leftarrow \mathsf{Bit2A}([\![b]\!]^{\mathsf{B}}, \lambda)$
Input: A Boolean secret sharing $[\![b]\!]^{\mathsf{B}}$ where $b \in \{0,1\}$.
Output: An arithmetical secret sharing $[\![b]\!]^{\mathsf{A}}$ where $b \in \{0,1\}$ is represented in $\mathbb{Z}_{2^\ell}$.

Setup phase:
1: For each $i \in \{0,1,2\}$, P_i samples random seeds $R_i \in \{0,1\}^\lambda$ and share it with P_{i-1}.
2: P_0 and P_1: $\alpha_1 \leftarrow \mathsf{Prg}(R_1, \ell)$
3: P_0: $\gamma \leftarrow_\$ \mathbb{Z}_2, \alpha_2 \leftarrow \gamma - \alpha_1$; Sends α_2 to P_2. ▷ Now $\mathsf{P}_1, \mathsf{P}_2$ *resp.* hold $(\alpha_1, \alpha_2 = \gamma - \alpha_1)$.
4: P_1, P_2: $\theta \leftarrow_\$ \{0,1\}$
5: P_1, P_2: Let $[v]^{\mathsf{A}} := ((-1)^\theta \cdot \alpha_1, (-1)^\theta \cdot \alpha_2)$
6: $[\![v]\!]^{\mathsf{A}} \leftarrow \mathsf{ShareConvert}(\mathcal{K}, [v]^{\mathsf{A}})$

Online phase:
7: P_0 sends $m_0 = b_0 \oplus b_1 \oplus \gamma$ to both P_1 and P_2, upon receiving m_0 then both P_1 and P_2 compute

$$\beta = m_0 \oplus b_2 = b \oplus \gamma$$
$$\sigma = \beta \oplus \theta$$

8: P_1 or P_2 sends $m_1 = \theta \oplus b_2$ to P_0, upon receiving m_1 then P_0 computes

$$\sigma = m_1 \oplus (b_0 \oplus b_1) \oplus \gamma$$

9: Let $[\![\beta]\!]^{\mathsf{A}} := \{(0,0), (0,\beta), (\beta,0)\}$, then $\mathsf{P}_0, \mathsf{P}_1, \mathsf{P}_2$ locally compute:

$$[\![v^*]\!]^{\mathsf{A}} \leftarrow \mathsf{RSS.sMul}((-1)^\sigma, [\![v]\!]^{\mathsf{A}})$$
$$[\![b]\!]^{\mathsf{A}} \leftarrow \mathsf{RSS.add}([\![v^*]\!]^{\mathsf{A}}, [\![\beta]\!]^{\mathsf{A}})$$

10: Outputs $[\![b]\!]^{\mathsf{A}}$.

We formally present the **correctness** and **security** of Π_1 respectively in the following.

Theorem 1. *In Π_1, for $\gamma, \theta \leftarrow_\$ \{0,1\}$, with $\forall b \in \{0,1\}$, assuming all parties honestly follow the protocol description,*

the final output $[\![b]\!]^{\mathsf{A}}$ is equal to the replicated secret sharing of b.

Proof. Table II lists the whole truth table of Π_1, including all variables in both the pre-processing and online phases. From which it shows $[\![v^*]\!]^{\mathsf{A}} + [\![\beta]\!]^{\mathsf{A}}$ in the last column aligns with b's replicated secret sharing. □

Theorem 2. *Assuming all parties honestly follow the protocol description in Π_1, i.e., $\gamma, \theta \leftarrow_{\$} \{0,1\}$, at most one party in $\{\mathsf{P}_0, \mathsf{P}_1, \mathsf{P}_2\}$ being controlled by an adversary $\mathcal{A}$, we claim that Π_1 computes $\mathcal{F}_{\mathsf{Bit2A}}$ with perfect security in the $\mathcal{F}_{\mathsf{CR}}$-hybrid model.*

Proof. First note that the setup phase does not involve any information about b, and the 3 parties share a random value v that is not connected to b. Later, in the online phase, the value $\beta = b \oplus \gamma$ recieved by $\{\mathsf{P}_1, \mathsf{P}_2\}$ is masked by $\gamma \leftarrow_{\$} \{0,1\}$, and the value $\sigma = \theta \oplus b \oplus \gamma$ recieved by P_0 is masked by $\theta \leftarrow_{\$} \{0,1\}$. Therefore, as long as the parties choose γ and θ in a uniformly random manner and keep them confidential, Π_1 is information-theoretically secure in the semi-honest setting. □

Efficiency. The setup phase of Π_1 incurs a total transmission cost of 4ℓ bits, which includes ℓ bits for the preparation of $[v]^{\mathsf{A}}$ (line 3), and an additional 3ℓ bits when invoking the ShareConvert function (line 6). Meanwhile, the online phase of Π_1 is considerably more efficient, requiring only total 3 bits. This involves P_1 or P_2 sending a masked bit m_1 to P_0, while P_0 sends a masked bit m_0 to both P_1 and P_2.

Compared to the existing Bit2A [3] protocol in ABY3 in the 3PC setting, which necessitates a total of 6ℓ bits in the online phase our method significantly outperforms by requiring only 4ℓ bits in the setup phase and 3 bits in the online phase. Furthermore, this enhanced efficiency is achieved without compromising the optimized one-round communication cost, marking a substantial improvement over the previous method.

B. B2A *Protocol*

Given a Boolean secret sharing $[\![x]\!]^{\mathsf{B}}$ where $x \in \{0,1\}^\ell$, directly applying Π_1 for Bit2A conversion on each $x[i] (i \in [\ell])$ and accumulating all scaled results as demonstrated in Eq. 3, we achieve a round-optimized B2A protocol. This protocol requires only one round in the online phase, since all scalar multiplication and addition operations involved can be executed locally. However, on the other hand this naive approach incurs a communication volume cost that is ℓ times that of Π_1, *i.e.,* $4\ell^2/3$ bits per party and ℓ bits per party in the setup and online phases respectively.

An interesting question we investigate is how can we reduce the overall communication overhead of the above naive approach? Here, $\forall i \in [\ell]$, when computing $[\![v_i]\!]^{\mathsf{A}}$ in the above naive approach, observe that in the setup phase if we generate only the 2-out-of-2 arithmetic secret sharing of $[v_i]^{\mathsf{A}}$ held by P_1 and P_2, skipping the ShareConvert step (in the used above naive approach), then it suffices to compute $[\![x]\!]^{\mathsf{A}}$ from $[\![x]\!]^{\mathsf{B}}$ with $\{[v_i]^{\mathsf{A}}\}_{i\in[\ell]}$ in two rounds in the later online phase, *i.e.,* the first round to compute $[x]^{\mathsf{A}}$ and a second round converting $[x]^{\mathsf{A}}$ to $[\![x]\!]^{\mathsf{A}}$ using ShareConvert.

Thus, to further reduce the communication cost in the aforementioned naive protocol, we propose a B2A protocol Π_2 detailed in Protocol 2 that reduces the total communication cost from $4\ell^2/3+\ell$ bits per party to $(\ell^2+5\ell)/3$ bits per party, at the cost of two rounds in the online phase.

Protocol 2 B2A potocol (Π_2)

Functionality: $[\![x]\!]^{\mathsf{A}} \leftarrow \mathsf{B2A}([\![x]\!]^{\mathsf{B}})$

Input: A Boolean secret sharing $[\![x]\!]^{\mathsf{B}}$ where $x \in \{0,1\}^\ell$, where for each $i \in [\ell]$ that each party $\mathsf{P}_j (j \in [3])$ holds the replicated boolean secret sharing of $x[i]$ which is denoted as $x_{i,j}, x_{i,j+1}$.

Output: An arithmetical secret sharing $[\![x]\!]^{\mathsf{A}}$ where $x \in \{0,1\}^\ell$ is represented in $\mathbb{Z}_{2^\ell}$.

Setup phase:

1: For each $i \in [3]$, P_i samples a random seed $R_i \in \{0,1\}^\lambda$ and share it with P_{i-1}.

2: **for** $i = 0$ to $\ell - 1$ **do**

3: P_0 and P_1: $\alpha_{i,1} \leftarrow \mathsf{Prg}(R_1, \ell)$

4: P_0: $\gamma_i \leftarrow_{\$} \{0,1\}, \alpha_{i,2} \leftarrow \gamma_i - \alpha_{i,1}$; Sends $\alpha_{i,2}$ to P_2. ▷ P_0 obtains γ_i, and $\mathsf{P}_1, \mathsf{P}_2$ obtain $[\gamma_i]^{\mathsf{A}}$.

5: **end for**

Online phase:

6: $\mathsf{P}_1, \mathsf{P}_2$: Let $[v]^{\mathsf{A}} := (0,0)$.

7: **for** $i = 0$ to $\ell - 1$ **do**

8: P_0 sends $m_i = x_{i,0} \oplus x_{i,1} \oplus \gamma_i$ to both P_1 and P_2, upon receiving m_i then both P_1 and P_2 sequentially compute

$$\beta_i = m_i \oplus x_{i,2} = x[i] \oplus \gamma_i$$
$$[x[i]]^{\mathsf{A}} \leftarrow \mathsf{SS.add}(\beta_i, \mathsf{SS.sMul}((-1)^{\beta_i}, [\gamma_i]^{\mathsf{A}}))$$
$$[v]^{\mathsf{A}} \leftarrow \mathsf{SS.add}([v]^{\mathsf{A}}, \mathsf{SS.sMul}(2^{\ell-1-i}, [x[i]]^{\mathsf{A}}))$$

9: **end for**

10: $[\![v]\!]^{\mathsf{A}} \leftarrow \mathsf{ShareConvert}(\mathcal{K}, [v]^{\mathsf{A}})$

11: Outputs $[\![v]\!]^{\mathsf{A}}$.

In the following, we formally present the **correctness** and **security** of Π_2.

Theorem 3. *In Π_2, assuming all parties honestly follow the protocol description, i.e., $\forall i \in [\ell]$, P_0 selects $\gamma_i \leftarrow_{\$} \{0,1\}$, later in the online phase when given $\forall x \in \{0,1\}^\ell$, P_1 and P_2 compute $[\![v]\!]^{\mathsf{A}}$ as described, then the final output $[\![v]\!]^{\mathsf{A}}$ is equal to the replicated secret sharing of x.*

Proof. Notice that $\forall i \in [\ell]$, by applying Eq. 1 over $[\![x[i]]\!]^{\mathsf{B}}$, we have

$$x[i] = (-1)^{\beta_i} \cdot \gamma_i + \beta_i$$

for $\beta_i = x[i] \oplus \gamma_i$, also from definition we have

$$x = \sum_{i=0}^{\ell-1} 2^{\ell-1-i} \cdot x[i].$$

Thus, in Π_2 (after lines 7-9) the computed $[v]^{\mathsf{A}}$ is equal to the 2-out-of-2 secret sharing of x, which means the final output $[\![v]\!]^{\mathsf{A}}$ (after conversion by line 10) is as desired. □

Theorem 4. *Assuming all parties honestly follow the protocol as described in Π_2, i.e., $\forall i \in [\ell]$ that $\gamma_i \leftarrow_{\$} \{0,1\}$, at most one party in $\{\mathsf{P}_0, \mathsf{P}_1, \mathsf{P}_2\}$ being controlled by an adversary*

$\mathcal{A}$, we claim that Π_2 computes $\mathcal{F}_{\mathsf{B2A}}$ with perfect security in the $\mathcal{F}_{\mathsf{CR}}$-hybrid model.

Proof. First in the setup phase, for each $i \in [\ell]$, P_0 distributes to P_1 and P_2 an arithmetic secret sharing of γ_i independent of $x[i]$. No addtional information about x is revealed. Later, in the online phase, the revealed bit $x[i] \oplus \gamma_i$ perfectly conceals $x[i]$. Thus, as long as P_0 selects γ_i for each $i \in [\ell]$ uniformly at random and keeps them confidential, Π_2 is information theoretical secure in the semi-honest setting. □

Efficiency. In the setup phase, there is a total communication cost of ℓ^2; In the online phase, it incurs a cost of 2ℓ bits in the first round, and 3ℓ bits when running $\mathsf{ShareConvert}(\mathcal{K}, [x]^{\mathsf{A}})$, resulting in a total communication cost of 5ℓ bits.

IV. Future work

In our Bit2A protocol Π_1, part of the communication overhead arises from the secret sharing of a random bit γ, which necessitates an ℓ-bit size random message sent from P_0 to P_2. Similarly, in our B2A protocol Π_2, the principal communication cost stems from the secret sharing of each γ_i where $i \in [\ell]$, cumulatively resulting in ℓ^2 bits communication overhead in the setup phase. A pertinent question that emerges here: Is it possible to reduce the communication cost derived from the secret sharing of a random bit? How can we generate multiple instances of secret sharing of a random bit with minimal communication cost?

We observe that the notion of a pseudorandom correlated generator (PCG) was proposed in [15], with the aim of facilitating multiple instances of correlated random tuples derived from local expansion over a jointly held short seed. Nonetheless, the correlated tuple created by their PCG construction is not directly compatible with our use case. Moving forward, our goal is to investigate this direction further to discover applicable efficient methods for generating arithmetic secret sharing of a random bit.

References

[1] D. Demmler, T. Schneider, and M. Zohner, "ABY - A framework for efficient mixed-protocol secure two-party computation," in *NDSS 2015*. The Internet Society, Feb. 2015.

[2] A. Patra, T. Schneider, A. Suresh, and H. Yalame, "ABY2.0: improved mixed-protocol secure two-party computation," in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, M. Bailey and R. Greenstadt, Eds. USENIX Association, 2021, pp. 2165–2182. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/patra

[3] P. Mohassel and P. Rindal, "ABY^3: A mixed protocol framework for machine learning," in *ACM CCS 2018*, D. Lie, M. Mannan, M. Backes, and X. Wang, Eds. ACM Press, Oct. 2018, pp. 35–52.

[4] D. Rathee, M. Rathee, N. Kumar, N. Chandran, D. Gupta, A. Rastogi, and R. Sharma, "Cryptflow2: Practical 2-party secure inference," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 325–342.

[5] Z. Huang, W.-j. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure two-party deep neural network inference," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 809–826.

[6] S. Addanki, K. Garbe, E. Jaffe, R. Ostrovsky, and A. Polychroniadou, "Prio+: Privacy preserving aggregate statistics via boolean shares," in *Security and Cryptography for Networks: 13th International Conference, SCN 2022, Amalfi (SA), Italy, September 12–14, 2022, Proceedings*. Springer, 2022, pp. 516–539.

[7] G. Asharov, Y. Lindell, T. Schneider, and M. Zohner, "More efficient oblivious transfer and extensions for faster secure computation," in *ACM CCS 2013*, A.-R. Sadeghi, V. D. Gligor, and M. Yung, Eds. ACM Press, Nov. 2013, pp. 535–548.

[8] D. Rotaru and T. Wood, "MArBled circuits: Mixing arithmetic and Boolean circuits with active security," in *INDOCRYPT 2019*, ser. LNCS, F. Hao, S. Ruj, and S. Sen Gupta, Eds., vol. 11898. Springer, Heidelberg, Dec. 2019, pp. 227–249.

[9] A. Aly, E. Orsini, D. Rotaru, N. P. Smart, and T. Wood, "Zaphod: Efficiently combining LSSS and garbled circuits in SCALE," Cryptology ePrint Archive, Report 2019/974, 2019, https://eprint.iacr.org/2019/974.

[10] D. Rotaru, N. P. Smart, T. Tanguy, F. Vercauteren, and T. Wood, "Actively secure setup for SPDZ," Cryptology ePrint Archive, Report 2019/1300, 2019, https://eprint.iacr.org/2019/1300.

[11] C. Bonte, N. P. Smart, and T. Tanguy, "Thresholdizing HashEdDSA: MPC to the rescue," Cryptology ePrint Archive, Report 2020/214, 2020, https://eprint.iacr.org/2020/214.

[12] D. Escudero, S. Ghosh, M. Keller, R. Rachuri, and P. Scholl, "Improved primitives for MPC over mixed arithmetic-binary circuits," in *CRYPTO 2020, Part II*, ser. LNCS, D. Micciancio and T. Ristenpart, Eds., vol. 12171. Springer, Heidelberg, Aug. 2020, pp. 823–852.

[13] A. Baccarini, M. Blanton, and C. Yuan, "Multi-party replicated secret sharing over a ring with applications to privacy-preserving machine learning," *Cryptology ePrint Archive*, 2020.

[14] T. Araki, J. Furukawa, Y. Lindell, A. Nof, and K. Ohara, "High-throughput semi-honest secure three-party computation with an honest majority," in *ACM CCS 2016*, E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, Eds. ACM Press, Oct. 2016, pp. 805–817.

[15] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, L. Kohl, and P. Scholl, "Efficient pseudorandom correlation generators: Silent ot extension and more," in *Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39*. Springer, 2019, pp. 489–518.